```
"""
calibrate.py — derive closure thresholds from actual GENERIC trajectory statistic

Run this once to calibrate SemanticFrictionConfig for a given UMA Config.

Usage::
    from uma.semantic.calibrate import calibrate_closure
    thresholds = calibrate_closure(cfg, n_traj=50, n_steps=3000, burnin=1000)
    print(thresholds)

Returns a dict you paste directly into SemanticFrictionConfig.
"""

import numpy as np
import math


def calibrate_closure(cfg, n_traj=50, n_steps=3000, burnin=1000,
                      dt=0.04, seed_start=0, verbose=True):
    """
    Run n_traj independent GENERIC trajectories and derive:
        E_target   — mean DC mode energy at equilibrium
        E_tol      — std of DC mode energy (natural fluctuation amplitude)
        dz_dt_floor — mean |dE/dt| at equilibrium (thermal noise rate)
        field_scale — 2 * sqrt(2 * E_target)  (natural |z| scale)

    These replace any hand-chosen or guessed constants.
    """
    from uma import UMAClient
    from uma.core.state import FieldPosterior

    N = cfg.grid.N
    kT = cfg.generic.kT

    all_E = []
    all_dz_dt = []

    for seed in range(seed_start, seed_start + n_traj):
        rng = np.random.default_rng(seed)
        client = UMAClient(cfg)
        psi0 = 0.3 * (rng.standard_normal((N, N)) +
                      1j * rng.standard_normal((N, N)))
        client.initialize(psi0, sigma0=0.05, dt=dt)
```

```
    E_prev = None
    for step in range(n_steps):
        psi = client.projection.lift(client.posterior_state.mean)
        psi = client.dynamics.step(psi, dt, rng=rng)
        z = client.projection.project(psi)
        client.posterior_state = FieldPosterior.full(
            z, client.posterior_state.Sigma)

        if step >= burnin:
            E = float(np.real(z @ z) / 2.0)
            if E_prev is not None:
                all_dz_dt.append(abs(E - E_prev) / dt)
            all_E.append(E)
            E_prev = E

    if verbose and (seed - seed_start) % 10 == 9:
        print(f'  ... {seed - seed_start + 1}/{n_traj} trajectories done')

all_E = np.array(all_E)
all_dz_dt = np.array(all_dz_dt)

E_target    = float(all_E.mean())
E_tol       = float(all_E.std())
dz_dt_floor = float(all_dz_dt.mean())
dz_dt_p25   = float(np.percentile(all_dz_dt, 25))
field_scale = float(2.0 * math.sqrt(2.0 * E_target))

thresholds = {
    # Paste these into SemanticFrictionConfig
    'E_target':           E_target,
    'E_tol':              E_tol,
    'dz_dt_floor':        dz_dt_floor,   # closure when |dE/dt| < this
    'dz_dt_floor_tight':  dz_dt_p25,     # tighter: lower quartile
    'field_scale':        field_scale,
    # Derived ratios
    'E_target_over_kT':   E_target / kT,
    'E_tol_over_kT':      E_tol / kT,
    'n_samples':          len(all_E),
    'n_traj':             n_traj,
    'kT':                 kT,
    'N':                  N,
    'dt':                 dt,
}

if verbose:
    print()
    print('=== Calibrated GENERIC closure thresholds ===')
```

```
        print(f'  E_target   = {E_target:.4e}  ({E_target/kT:.3f} kT)')
        print(f'  E_tol      = {E_tol:.4e}  ({E_tol/kT:.3f} kT)')
        print(f'  dz_floor   = {dz_dt_floor:.4e}  (mean thermal rate)')
        print(f'  dz_floor25 = {dz_dt_p25:.4e}  (lower quartile)')
        print(f'  field_scale= {field_scale:.4e}')
        print()
        print('Paste into SemanticFrictionConfig:')
        print(f'  convergence_dz_dt = {dz_dt_floor:.4e}')
        print(f'  closure_state_tol = {field_scale:.4e}')

    return thresholds


def make_friction_config(cfg, dt=0.04, n_traj=50, **kwargs):
    """
    Build a fully calibrated SemanticFrictionConfig from GENERIC statistics.
    """
    from uma.semantic.friction import SemanticFrictionConfig

    thresholds = calibrate_closure(cfg, dt=dt, n_traj=n_traj,
                                   verbose=kwargs.pop('verbose', True))
    return SemanticFrictionConfig(
        kT=cfg.generic.kT,
        lam=cfg.generic.lam,
        N=cfg.grid.N,
        dt=dt,
        convergence_dz_dt=thresholds['dz_dt_floor'],
        closure_state_tol=thresholds['field_scale'] * 2.0,
        **kwargs,
    ), thresholds
```